

Privora

On-Premises Deployment Guide

Prepared for: Bassam — Founder, Privora
April 2026 | Confidential

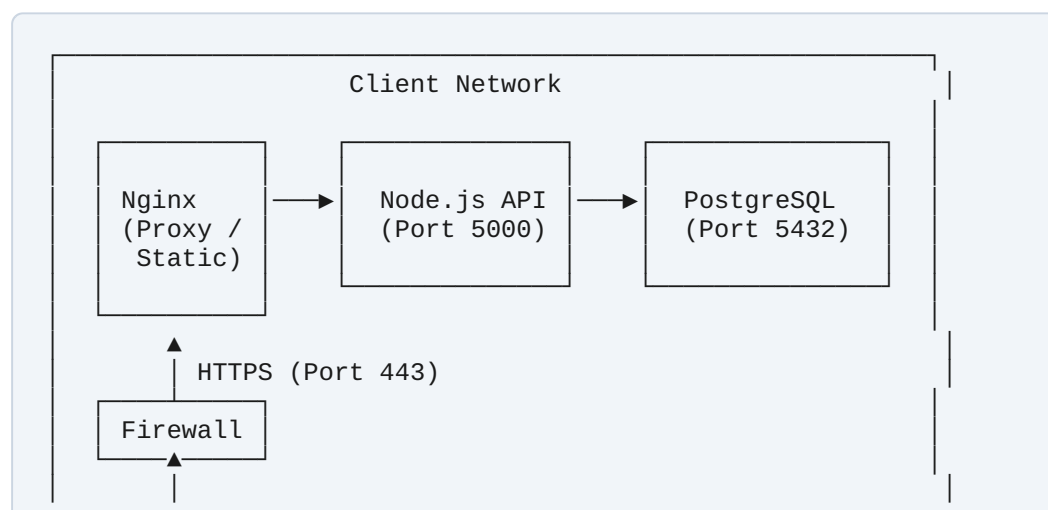
Privora — On-Premises Deployment Guide

Purpose

This guide covers deploying Privora entirely within a client's own infrastructure — whether on dedicated servers, a private cloud (VMware, OpenStack), or a managed private cloud (AWS VPC, Azure Private Cloud).

This is the deployment model for enterprise clients and government institutions that require data to remain within their own network.

1. Architecture for On-Premises





Internet / Intranet users

All components run inside the client's network. No data leaves the premises.

2. Infrastructure Requirements

Minimum (up to 20 organizations, ~1,000 users)

Resource	Specification
CPU	4 vCPU (Intel/AMD x86-64)
RAM	8 GB
Storage	100 GB SSD
OS	Ubuntu 22.04 LTS
Docker	24.x
Docker Compose	2.x
Network	1 Gbps internal, 100 Mbps internet (for email delivery)

Recommended (up to 100 organizations, ~10,000 users)

Resource	Specification
CPU	8 vCPU
RAM	16 GB
Storage	500 GB SSD (RAID-1 or equivalent)
OS	Ubuntu 22.04 LTS
Separate DB Server	Optional — 4 vCPU, 8 GB RAM, 200 GB SSD

Enterprise / Government (100+ organizations)

Component	Specification
Application servers	2× 8 vCPU, 16 GB RAM (load balanced)
Database server	8 vCPU, 32 GB RAM, 1 TB NVMe SSD
Load balancer	Nginx or HAProxy
Backup storage	Separate NAS or S3-compatible storage

3. Required Ports

Port	Protocol	Direction	Service
443	HTTPS	Inbound (internet-facing)	Frontend + API (via Nginx)
80	HTTP	Inbound	Redirect to 443
5000	HTTP	Internal only	Node.js API
5432	TCP	Internal only	PostgreSQL
587	TCP	Outbound	SMTP (email delivery)
22	SSH	Admin only	Server management

Firewall Rules: - Ports 5000 and 5432 must NOT be accessible from outside the server/container network - Port 443 is the only internet-facing port - Port 587 outbound is required only if using external SMTP (SendGrid/AWS SES) - For fully isolated networks: deploy a local SMTP server (Postfix/Mailhog)

4. Step-by-Step Installation

Step 1 — Prepare the Server

```
# Update OS
sudo apt update && sudo apt upgrade -y

# Install Docker
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# Install Docker Compose
sudo apt install docker-compose-plugin -y

# Verify
docker --version
docker compose version
```

Step 2 — Clone Repository

```
# Option A: From GitHub (requires internet access)
git clone https://github.com/0xBassam/Bassam.git /opt/privora

# Option B: From a USB / internal server (air-gapped)
# Copy the project folder to /opt/privora
```

Step 3 — Configure Environment

```
cd /opt/privora

# Create environment file
cp .env.docker .env

# Edit with your values
nano .env
```

Required values for on-premises:

```
# Database
DB_PASSWORD=your_strong_db_password

# JWT Secrets (generate: openssl rand -hex 64)
JWT_SECRET=your_64_char_jwt_secret_here
JWT_REFRESH_SECRET=your_64_char_refresh_secret_here

# OTP
OTP_SECRET=your_otp_signing_secret

# Email (use your own SMTP or set up a local mail server)
SMTP_HOST=your.smtp.server.com
SMTP_PORT=587
SMTP_USER=privora_emailuser
SMTP_PASS=your_smtp_password
EMAIL_FROM=no-reply@yourclientdomain.com

# Set these to your actual domain or server IP
CLIENT_URL=https://privora.yourclientdomain.com
VITE_API_URL=https://privora.yourclientdomain.com/api/v1
```

Step 4 — SSL Certificate

Option A: Own domain with Let's Encrypt

```
sudo apt install certbot -y
sudo certbot certonly --standalone -d
privora.yourclientdomain.com
# Certificates stored at: /etc/letsencrypt/live/
privora.yourclientdomain.com/
```

Update `docker-compose.yml` frontend service to mount the certificates:

```
frontend:
  volumes:
    - /etc/letsencrypt:/etc/letsencrypt:ro
```

Option B: Self-signed certificate (internal networks only)

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /opt/privora/certs/privora.key \
```

```
-out /opt/privora/certs/privora.crt \  
-subj "/CN=privora.internal"
```

Option C: Client provides their own SSL certificate

Copy the certificate and key to `/opt/privora/certs/` and reference them in the Nginx configuration.

Step 5 — Build and Start

```
cd /opt/privora  
  
# Build all Docker images  
docker compose build  
  
# Start all services  
docker compose up -d  
  
# Verify all containers are running  
docker compose ps
```

Expected output:

NAME	STATUS	PORTS
privora_db	running	127.0.0.1:5432->5432/tcp
privora_backend	running	127.0.0.1:5000->5000/tcp
privora_frontend	running	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp

Step 6 — Verify Installation

```
# API health check  
curl https://privora.yourclientdomain.com/api/v1/health  
  
# Expected  
{  
  "success": true,  
  "data": {  
    "status": "ok"  
  }  
}
```

Open `https://privora.yourclientdomain.com` in a browser. The landing page should load.

Step 7 — Create First Organization

```
curl -X POST https://privora.yourclientdomain.com/api/v1/auth/  
register-org \  
-H "Content-Type: application/json" \  
-d '{  
  "organizationName": "Client Organization Name",  
  "adminName": "Admin Name",  
  "adminEmail": "admin@clientorg.com"  
}'
```

The admin will receive an OTP to their email to complete setup.

5. Auto-Start on Server Reboot

```
# Create systemd service
sudo nano /etc/systemd/system/privora.service
```

```
[Unit]
Description=Privora Platform
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/privora
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
TimeoutStartSec=300

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable privora
sudo systemctl start privora
```

6. Backup Configuration

Automated Daily Backup

```
# Create backup script
nano /opt/privora/scripts/backup.sh
```

```
#!/bin/bash
BACKUP_DIR=/opt/privora/backups
DATE=$(date +%Y%m%d-%H%M)
mkdir -p $BACKUP_DIR

# Backup PostgreSQL
docker compose -f /opt/privora/docker-compose.yml exec -T db \
  pg_dump -U privora_user privora > $BACKUP_DIR/db-$DATE.sql

# Compress
gzip $BACKUP_DIR/db-$DATE.sql

# Keep only last 30 days
find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete

echo "Backup completed: db-$DATE.sql.gz"
```

```
chmod +x /opt/privora/scripts/backup.sh
```

```
# Schedule daily at 2 AM
echo "0 2 * * * root /opt/privora/scripts/backup.sh >> /var/log/
privora-backup.log 2>&1" \
| sudo tee /etc/cron.d/privora-backup
```

7. Air-Gapped Deployment (No Internet)

For networks with no internet access:

Step 1 — Pre-pull Docker images on an internet-connected machine:

```
docker pull node:20-alpine
docker pull postgres:15-alpine
docker pull nginx:alpine
docker save node:20-alpine postgres:15-alpine nginx:alpine |
gzip > privora-images.tar.gz
```

Step 2 — Transfer the `.tar.gz` file and the project folder to the air-gapped server via USB or internal file transfer.

Step 3 — Load images on the air-gapped server:

```
docker load < privora-images.tar.gz
```

Step 4 — Build application images from source (no internet needed as base images are loaded):

```
cd /opt/privora
docker compose build
docker compose up -d
```

Email in air-gapped environments: Deploy a local SMTP server (Postfix) or use a local mail relay. Set `SMTP_HOST` to the internal mail server address.

8. Updating the Application (On-Premises)

With internet access:

```
cd /opt/privora
git pull origin main
docker compose build
docker compose up -d
```

Without internet access: 1. Build new Docker images on an internet-connected machine 2. Export: `docker save privora-backend privora-frontend | gzip`

> update.tar.gz 3. Transfer to air-gapped server 4. Load: `docker load < update.tar.gz` 5. Restart: `docker compose up -d`

9. Security Hardening Checklist

Before handing the system to a client, complete these steps:

- ☐ Change all default passwords in `.env`
 - ☐ Generate unique JWT_SECRET and JWT_REFRESH_SECRET (minimum 64 characters)
 - ☐ Ensure ports 5000 and 5432 are not accessible externally
 - ☐ Enable firewall (`ufw enable`, allow 22/443/80 only)
 - ☐ Enable automatic OS security updates: `sudo apt install unattended-upgrades`
 - ☐ Set up log rotation for Docker logs
 - ☐ Configure SSL certificate with a valid CA (not self-signed for production)
 - ☐ Disable SSH root login: set `PermitRootLogin no` in `/etc/ssh/sshd_config`
 - ☐ Set up automated daily backups with offsite copy
 - ☐ Test backup restore procedure before go-live
 - ☐ Configure Fail2ban to block brute-force SSH attempts
-

10. Monitoring On-Premises

Since Railway/Vercel dashboards aren't available, set up local monitoring:

Simple Uptime Check

```
# Add to crontab – alert if API is down
*/5 * * * * curl -sf https://privora.internal/api/v1/health || \
  echo "Privora is DOWN" | mail -s "ALERT: Privora Down"
admin@client.com
```

Docker Log Monitoring

```
# Stream all logs
docker compose logs -f
```



```
# View last 100 lines of backend
docker compose logs --tail=100 backend

# Log rotation (prevent disk fill)
# Add to /etc/docker/daemon.json:
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "50m",
    "max-file": "5"
  }
}
```

| Support

For on-premises deployment support, contact the Privora team with: - Server OS version and specs - Output of `docker compose ps` - Backend logs: `docker compose logs --tail=200 backend` - Any error messages encountered